Beyond Mining Evolutions with Diffs: Interactive Search of Code Evolutions in Software Repositories

Quentin Le Dilavrec[1], Carolin Brandt[1] and Andy Zaidman[1]

[1]TU Delft

Abstract

Over multiple decades, software evolution has been studied through diff-based version control systems (e.g., Subversion). This paper starts from the hypothesis that we should not follow the same methodology to study software evolution in snapshot-based version control systems (e.g., Git), that are much more centered around collaborative practices. We propose an approach which fundamentally revises the study of software evolution by processing source code histories in a snapshot-first way, enabling the interactive search of code evolution in software repositories

Keywords

software evolution, software engineering, source code histories, search-based engineering

1. Introduction

The way we study software evolution in source code histories has not changed much since software engineers were mostly using *CVS* [1] and *SVN* [2]. But collaborative (*e.g.,* pull-based [3]) development using *Git* [4] has exploded, with millions of users and repositories on *GitHub*. This paper starts from the hypothesis that we cannot follow the same methodology to study software evolution in version control systems that are much more centered around collaborative practices (*i.e.,* snapshot-based, *e.g.,* Git), versus the version control systems that we used a decade ago or more (diff-based). More specifically, the use of diffs, as a systematic step in the process of finding evolutions, which could be called a diff-first approach. Indeed, diffs are considered a commodity, without properly considering their limits as information retrieval heuristics.

We propose an approach which fundamentally revises the study of software evolution by processing source code histories in a snapshot-first way. Our snapshot-first approach turns this around: given the pattern of the code evolution, we search the matching code instances in snapshots, then we compare them, only computing a diff if the code elements have changed location or content. By prioritizing code, and pushing back computing diffs to later in the process, our hypothesis radically reduces runtime of the historical analyses, thus, offering the opportunity of searching evolutions between branches and forks, and applying more expensive diff heuristics.

To implement our scalable snapshot-first approach, we propose an incremental code query system, whereby precomputing sub-queries, queries executed on entire snapshots (i.e., Git commits) can skip code elements that will not contain matches, leading to a highly scalable code querying mechanism largely *independent of the size of the code base*. Subsequently, by comparing found code element in a latent space, we only compute diffs when the found elements have changed, enabling to efficiently compare many pairs of snapshots without having to compute diffs for every one of them, leading to an evolution search largely *independent of unrelated changes*, thus freeing resources for more accurate and expensive diff heuristics.

Preliminary results indicate that our implementation has a query response time proportional to the number of code evolution matches. Moreover, using our approach we were able to find occurrences of

Benevol'24: The 23rd Belgium-Netherlands Software Evolution Workshop Thu 21 — Fri 22 November 2024, Namur, Belgium

qledilavrec@tudelft.nl (Q. Le Dilavrec); c.e.brandt@tudelft.nl (C. Brandt); a.e.zaidman@tudelft.nl (A. Zaidman)

0000-0001-7511-8394 (Q. Le Dilavrec); 0000-0001-7623-1970 (C. Brandt); 0000-0003-2413-3935 (A. Zaidman)



© 2024 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

code evolutions merged from forks that would have probably been missed (or at least lost the intent) by only considering the main branch.

2. Snapshot-first Evolution Search

Our approach enables searching for evolutions by specifying the code patterns it contains (code dimension) and how it is transformed (temporal dimension). Our approach extends *HyperAST* [5] with our incremental code query system, to (1) find code that matches the patterns and (2) efficiently find when these code patterns evolve, without the need to compute diffs on every (consecutive) commit pair.

HyperAST [5] is a specialized syntax tree structure enabling efficient in-memory processing of many snapshots of source code. It relies on a Direct Acyclic Graph (DAG) structure in which unique code subtrees are deduplicated; this enables incremental computation of metadata, *i.e.,* once per unique subtree. To enable searching for code patterns from snapshots in *HyperAST*, we have repurposed Tree-Sitter[6], a syntax tree based highlighter, that posesseses an associated query system based on s-expressions — transparently mapping to syntax nodes — and additional syntax to define various properties on syntax nodes (see Figure 1a for an example query). To materialize our code search at scale, we have ensured Tree-Sitter queries can work in synergy with *HyperAST*, notably: (1) extending the query executor to enable incremental queries, (2) implementing specific visitors to give access to precomputed query verdicts, and (3) integrating a shallow query executor to the creation process of nodes in *HyperAST*.

Fundamentally, our code search aims to scale by reducing the amount of code traversed, only requiring to traverse nodes that lead to instances of the searched patterns. Consequently, the search cost should be proportional to the number of matches, while decorrelated from the total code size. *In this paragraph we assume that before traversing a snapshot, a set of queries have already been indexed and precomputed. This preprocessing is further described in Section 2.1.* Before executing a query, its query plan must be compiled: this takes the form of an oriented graph of filtering *steps* on syntax nodes (*i.e.,* a state machine). We further process this graph to plan how the precomputed verdicts will be used to skip subtrees that do not contain the required patterns. This graph processing consists of exhaustively searching for all mandatory sub-patterns (*i.e.,* not under a conditional quantifier) that correspond to precomputed queries. The query execution is done by traversing the snapshot depth-first, while maintaining *query states* at different point in the *step* graph (*i.e., states* can be: inserted when descending in a potential pattern; updated when a step is validated; duplicated when a multiple transitions are possible, removed when failing a mandatory filter *step* or reaching a dead-end, matched when reaching a terminal *step*). To realize our approach — skip descending into subtrees that will not yield any match —, we conditioned the descent to the absence of pending *query states* (*i.e.,* non-terminal state in step graph) and the presence of the mandatory precomputed verdicts.

```
(try_statement
  (block
    (expression_statement
      (method_invocation
      (identifier) (#EQ? "fail"))))
  (catch_clause))
```

(a) Example Query matching a try catch containing a call to a method named `fail`

```
(try_statement (block)
               (catch_clause) )

(method_invocation
  (identifier) (#EQ? "fail") )

(identifier) (#EQ? "fail")
```

(b) Some Sub-queries from the Example

Figure 1: Example Query

2.1. Precomputing Query Verdicts

We hypothesize that precomputing query verdicts is essential for efficient snapshot searches, by bubbling up absence of patterns deep in each subtree of code without having to descend inside them. The precomputing of verdicts is achieved efficiently by extending the preprocessing approach from Le Dilavrec *et al.* [5], where local metadata can be computed and persisted per unique node (*i.e.,* subtree in the DAG). The queries selected to be precomputed are first compiled once at the beginning, then during query execution, verdicts are aggregated (*i.e.,* union over verdicts) with verdicts from children, this propagating upward the presence of code patterns. In addition, because each node is preprocessed independently, the query executions can be made shallow, *i.e.,* query states are only added once, contrary to snapshot wide queries.

Efficiently storing the verdicts is also essential for a scalable approach. Indeed, every additional byte consume by the verdicts must improve the code search efficiency, without worsening memory accesses to the point where it would cancel the runtime benefits. By finely controlling the exact memory representation of verdicts, we can reduce their memory footprint on particular cases, *e.g.*,, when no queries are matched in a subtree (the absence of positive verdict is kept on a group of nodes with the same characteristics, *i.e.,* region-based memory management).

2.2. Temporal Dimension of the Evolution Search

Table 1: Change Distances, by detection capabilities and usage as latent index.

Edit Distance	Change Detection			Latent Index	References
	Ins/Del	Moved	Modified		
diff	✓	✓	✓	×	[7, 4, 8, 9]
file changes	✓	✓[a]	✓[a]	×	[4, 9]
subtree matches	✓	✓	✓	×	[8, 9]
count	✓	×	×	✓	Our approach
paths	✓	✓	×	✓[b]	
contents	✓	×	✓	✓[b]	
paths + contents	✓	✓	✓	✓[b]	

[a] only on files [b] must be hashed to have a fixed sized

The second dimension of the search focuses on the temporal aspect, whereby code elements are mapped between snapshots to identify relevant evolutions. This section describes how our approach further exploits the temporal redundancy and precomputability of source code, by caching for each snapshot a latent space index of found code elements. This caching can avoid the computation of diffs on pairs of searched commits that do not contain changes to the code patterns.

In our approach, we consider the temporal dimension of mining evolutions as an edit distance problem. Where given two states (*e.g.,* snapshots), we must find the edit script (*i.e.,* sequence of changes) leading from one state to the other, while minimizing a cost defined by an edit distance (string distance [7], tree distance [10]). The cost can be dependent on the type of change (*e.g.,* insertion, modification) and the (code) elements they involve, for example transforming 'a' to 'b' could be made more expensive than transforming 'f' to 'd', or deleting 'f'. With our approach, by defining code patterns, it is possible to define precisely an edit distance. Taking a practical example with *Git* where it is possible to obtain the changed files[1] (*e.g.,* Removed, Renamed, Moved file), using `git diff -M100% -B100% commit1..commit2`

[1] https://git-scm.com/docs/git-diff#Documentation/git-diff.txt---name-status; last visited Wed 18 Sep 2024. Noting that the tree-based implementation is not available through the public API and that `-M100%` and `-B100%` do not guaranty the computation of the content-based diff

to find file renames with 100% identical content (50% by default), would be equivalent in our approach to a query matching nodes of type file, *i.e.,* `(file)`, and defining the same associated distance (*i.e.,* same when sharing the same file path, otherwise same if sharing 100% content).

Table 1 presents edit distances, by capabilities of detecting certain change types and whether representable in a latent semantic index. Therein, **diffs** represent complete approaches that work at the textual level (*e.g.,* Myers), or tree based (*e.g.,* RefactoringMiner, Gumtree). These complete approaches can detect — to various degree of granularity — moves and modifications, but they must be compiled for each pair of snapshots. Next, **file changes** and **subtree matches** correspond to partial approaches designed to better scale on tree structures with precomputed intrinsic hashes (*e.g.,* Git). While **file changes** is limited to file level changes, **subtree matches** works on all nodes (down to a configurable height). But both still need to be computed pairwise. To avoid most of these expensive pairwise comparisons, we propose additional distances that can be computed on individual snapshots, then cheaply compared pairwise. **Count** corresponds to the number of elements matched for a given pattern, thus it will only detect changes when the total number of matches changes. **Paths** represents a set of strings then offsets — from the root of the snapshot — leading to each element. **Contents** represents a distance over the content of each element, *i.e.,* any kind of hash computed on the code element. **paths** and **contents** represent sets, thus to produce a constant size value their content is hashed. More complex evolutions can be specified by combining distances — and patterns —, *e.g.,* combining **paths** and **contents** enables to detect both moves and modifications — combining the patterns `(try_statement (catch_clause) .)` and `(try_statement (catch_clause). (finally_clause))` enables to detect the addition of a finally clause to a try statement with a catch clause.

3. Expected Benefits and Early Results

This section presents how our approach could benefit researchers and practitioners in their work. Indeed, to study and maintain software, difficult trade-offs have to be made (*e.g.,* precision vs time, distributed vs centralized). With our approach we expect to improve upon three major limitations: (1) going beyond the linear histories to handle highly collaborative project, (2) going beyond files to consider code in its tree form, and (3) going beyond batch processing to increase interactivity.

3.1. Beyond Linear Histories: considering evolutions from merged branches

Traditional approaches to studying software evolution may prefer considering development histories as being linear, such as when using *PyDriller* [11], where it is possible to select the main branch and ignore merges. Tools used to extract changes or refactorings have been evaluated on consecutive commits, ignoring merges [12, 8].

But considering histories as linear can lead to inaccuracies, and incomplete observations. In particular, Bertoncello *et al.* [13] expose the limitations of only considering commits on the main branch when measuring developer contribution. Similarly, Maj *et al.* [14] have found inconsistent timestamps in certain software histories, while studying the correlation between popularity and software maturity. Moreover, to investigate the developer intent behind changes observed on a merged commit, while only considering the main branch will require to both manually trace the source of the change and manually interpret the commit message [15].

Our snapshot-first approach is designed to efficiently find specific evolutions between pairs of commits. Thus, we can rely less on commit metadata (parents or commit message) to select pairs of commits. We can even consider long-running branches, and potentially even cherrypicking. With our implementation, we were able to find evolutions clearly exposing developer intent that would have been made more difficult to impossible to interpret if only commits from the main branch would have been considered (see Appendix B.1).

3.2. Beyond Changed Files: Filtering Fine-Grained Evolution Components

When attempting to find evolutions it is possible to filter-out unchanged files, *e.g.,* relying on *Git* to flag files as added, removed, modified, or renamed. This strategy is notably leveraged by tools such as, *RefactoringMiner* [12]; and provided by *Boa* [16]. This filtering enables scaling to large repositories, by decorrelating the cost of analysis from the total number of files (*i.e.,* only considering changed files). But once files have been obtained they still need to be further processed, increasing the methodological complexity of the approach. Moreover, using *Git* to find renamed files still requires diffing files contents if they have changed (even one character). And if too many files are changed between two commits, the cost of computing changes and analyzing the files might still be more than expected (*i.e.,* leading to unreliable scalability) (see Appendix A). Finally, only considering changed files limits possibilities of taking into account code path going through unchanged files [12].

With our approach, we can directly compare the found code elements, and the entirety of code is available as syntax nodes enriched with precomputed metadata. It would be possible to focus on specific co-change between individual code elements, *e.g.,* classes and methods. Our approach could also help developers browse large code histories, with many changed files, as our approach does not rely on file level change detection to efficiently find evolutions. To get an idea of the responsiveness of our approach we measured the average runtime necessary for find code matching pattern in Figure 1a, with the patterns in Figure 1b already precomputed (see Appendix B.2). Excluding *aws-sdk-java* and *aws-sdk-java-v2* that failed during *HyperAST* construction, and *Quarkus* and *Flink* that were timeout, we were able to find pattern occurrences in less than a second per snapshot. Notably, on Hadoop (1.61 Million lines of Java code) we were able to find between 2579 and 2600 occurrences in less than a second and on average 719ms.

4. Summary

We have presented an overview of our approach to search evolutions in software histories, by making code search leverage temporal code redundancy, and only computing diffs when found code elements have changed.

We also found potential use cases where our approach could improve the study and investigation of evolutions in software histories. Qualitatively, we used our implementation to interactively search for evolutions, at the AST level in highly collaborative projects, and found instances of squashed commits holding information about developer intent that would have been missed if only the main branch was considered. Quantitatively, we measured the code search time on individual snapshots and observed on Hadoop (1.61 Million lines of Java code) query runtime under a second to find between 2579 and 2600 occurrences per commit.

In terms of future work, we envision to experiment with different verdict representations, to precompute large number of queries, *e.g.,* using bloom filters or including captured text. We are also considering to try more latent code representations to find different types of changes. We could also further combine the subtree matching with the code search, to compute diffs faster. Another aspect to investigate is how efficient our approach is at find code clones. Finally, we will investigate to provide cross-reference capabilities, to enable both more general and precise queries, *e.g.,* using fully qualified names or matching both on declaration and their references.

References

[1] D. Grune, Concurrent versions system, 1990. URL: http://cvs.nongnu.org/, accessed: 2024-09-11.

[2] C. VersionOne, Apache subversion centralized version control, 2000. URL: http://subversion.apache.org/, accessed: 2024-09-11.

[3] G. Gousios, M. Pinzger, A. van Deursen, An exploratory study of the pull-based software develop-

ment model, in: Proceedings of the 36th international conference on software engineering, 2014, pp. 345–355.
[4] L. Torvalds, Git - fast version control, 2005. URL: https://git-scm.com/, accessed: 2024-09-11.
[5] Q. Le Dilavrec, D. E. Khelladi, A. Blouin, J.-M. Jézéquel, Hyperast: Enabling efficient analysis of software histories at scale, in: 37th IEEE/ACM International Conference on Automated Software Engineering, IEEE/ACM, 2022, pp. 1–12.
[6] M. Brunsfeld, Tree-sitter, https://tree-sitter.github.io/tree-sitter/, 2022. Accessed: 2022-05-06.
[7] E. W. Myers, An o(nd) difference algorithm and its variations, Algorithmica 1 (1986) 251–266.
[8] J. Falleri, F. Morandat, X. Blanc, M. Martinez, M. Monperrus, Fine-grained and accurate source code differencing, in: ACM/IEEE International Conference on Automated Software Engineering, ASE '14, ACM/IEEE, 2014, pp. 313–324.
[9] Q. Le Dilavrec, D. E. Khelladi, A. Blouin, J.-M. Jézéquel, Hyperdiff: Computing source code diffs at scale, in: 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'23), 2023.
[10] K. Zhang, D. Shasha, Simple fast algorithms for the editing distance between trees and related problems, SIAM journal on computing 18 (1989) 1245–1262.
[11] D. Spadini, M. Aniche, A. Bacchelli, Pydriller: Python framework for mining software repositories, in: Proceedings of the 2018 26th ACM Joint meeting on european software engineering conference and symposium on the foundations of software engineering, 2018, pp. 908–911.
[12] N. Tsantalis, M. Mansouri, L. M. Eshkevari, D. Mazinanian, D. Dig, Accurate and efficient refactoring detection in commit history, in: Proceedings of the 40th International Conference on Software Engineering, ACM, 2018, pp. 483–494.
[13] M. V. Bertoncello, G. Pinto, I. S. Wiese, I. Steinmacher, Pull requests or commits? which method should we use to study contributors' behavior?, in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2020, pp. 592–601.
[14] P. Maj, S. Muroya, K. Siek, L. Di Grazia, J. Vitek, The fault in our stars: Designing reproducible large-scale code analysis experiments, in: 38th European Conference on Object-Oriented Programming (ECOOP 2024), Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
[15] D. J. Kim, T.-H. Chen, J. Yang, The secret life of test smells-an empirical study on test smell evolution and maintenance, Empirical Software Engineering 26 (2021) 1–47.
[16] R. Dyer, H. A. Nguyen, H. Rajan, T. N. Nguyen, Boa: A language and infrastructure for analyzing ultra-large-scale software repositories, in: 2013 35th International Conference on Software Engineering (ICSE), IEEE, 2013, pp. 422–431.
[17] T. Winters, T. Manshreck, H. Wright, Software engineering at google: Lessons learned from programming over time, O'Reilly Media, 2020.

A. Quote From *Software Engineering At Google*

> "That said, given the rate of commits for machine-generated changes, naive 'blame' tracking has less value than it does in more change-averse ecosystems."
>
> — Titus Winters *et al. Software Engineering at Google (2020), p. 355*[17]

This quote highlights some limitations of blames, related to changes impacting many files (possibly thousands). We suppose it actually concerns a limitation of diff-first approaches optimized at the file level, when the content of too many files have changed, *i.e.,* the intrinsic ids of blobs cannot be used to quickly match files, thus diffs have to be computed between every changed files up to a practical limit, *i.e.,* will miss increasingly more files renames.

B. Early Results

B.1. Squashed Commits involving the code pattern in Figure 1a found with our approach

```
CustomWeighting: ensure Map is ordered (#2162)
* CustomWeighting: ensure Map is ordered

* minor

* cleaner junit5 assertThrows

* one more assertThrow
```

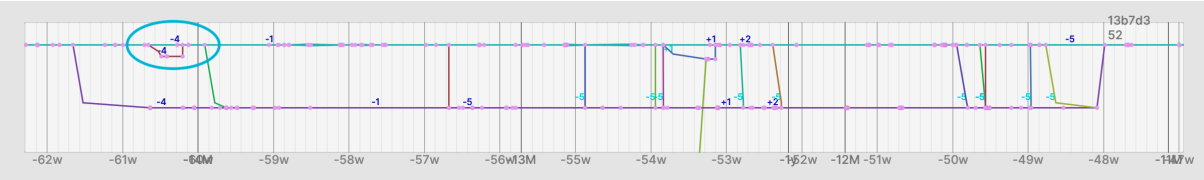
(a) merged commit message dd9dbd5

`CustomWeighting: ensure Map is ordered`

(b) pull request title #2162

`cleaner juni5 assertThrows`

(c) feature commit message c8ab97c



(d) Commit Graph with match counts

Figure 2: Pull Request from graphhopper/graphhopper #2163

```
Load EncodingManager from properties instead of config
```

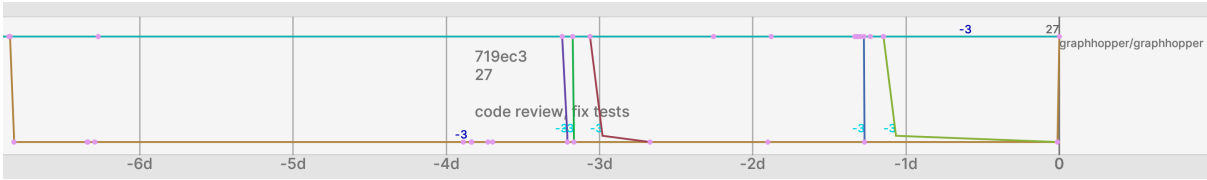
(a) merged commit message dd9dbd5

`Load EncodingManager from properties instead of config`

(b) pull request title #2607

`code review, fix tests`

(c) feature commit message 719ec34



(d) Commit Graph With Match Counts

Figure 3: Pull Request from graphhopper/graphhopper #2607

B.2. Code Search Runtime per Snapshot

Considering the projects from [9]. We measured the runtime of our query system over 1000 snapshots. We used the query from Figure 1a and the sub-queries from Figure 1b.

Table 2: Code Query (Figure 1a) Runtime per snapshot

		Runtime (ms)			Matches	
Projects	LoC	Mean	Min	Max	Min	Max
hadoop	1.63M	719	685	982	2,579	2,600
flink	1.50M	Timeout	Timeout	>1s	Timeout	>918
quarkus	614K	Timeout	Timeout	>1s	Timeout	>63
guava	509K	29	21	44	0	0
netty	317K	127	114	499	150	194
dubbo	197K	146	100	321	82	128
fastjson	188K	14	9	21	1	2
logging-log4j2	183K	177	157	339	19	23
jenkins	181K	54	48	170	19	120
javaparser	179K	22	20	84	9	14
spoon	154K	66	56	188	71	73
aws-toolkit-eclipse	93.9K	7	0	15	0	0
maven	92.5K	197	181	409	0	51
spark	85.6K	154	141	355	2	43
skywalking	84.7K	254	70	618	1	2
jackson-core	52.3K	11	5	17	45	168
arthas	44.2K	24	17	63	0	0
jacoco	38.8K	44	26	92	0	0
junit4	31.2K	13	8	120	28	44
gson	25.8K	28	21	242	160	304
slf4j	13.5K	16	13	34	3	8

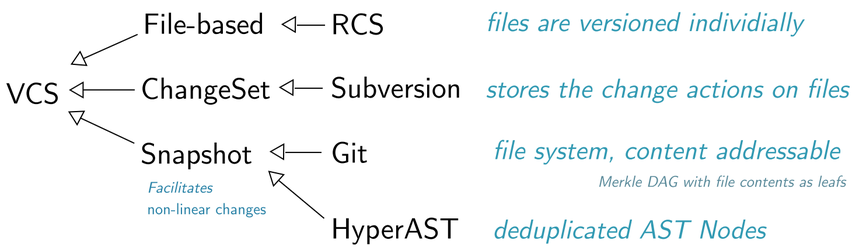
Beyond Mining Evolutions with Diffs:

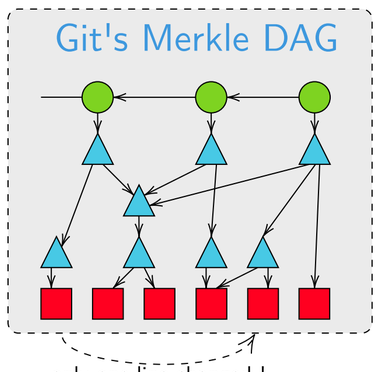
Interactive Search of Code Evolutions in Software Repositories

Quentin Le Dilavrec

in collaboration with Carolin Brandt and Andy Zaidman

Over multiple decades,
software evolution has been studied successfully
through diff-based version control systems (VCS), e.g., Subversion.





Hypothesis

New methodologies are needed to continue study software evolution
in snapshot-based version control systems (e.g., Git),
especially to study **collaborative** practices and **large** projects.

Approach

We propose an approach which fundamentally revises the extraction process of software evolutions by considering source code histories in a **snapshot-first** way, enabling the **interactive search of code evolutions** in software repositories.

Incremental code query system

1) incrementally precompute sub-queries on subtrees
2) skip code elements that will not contain matches
   when executing queries on entire commits

consequence: pattern search independent of the size of the code base,
   but still dependent on the number of matches

Finding changed code instances

1) use a latent space to represent the matched code element
2) compare the commits in this latent space,
3) if different in latent space, do further comparisons.

consequence: evolution search largely independent of unrelated changes,
- freeing resources for more accurate and expensive diff heuristics.
- changes of diff algorithms do not impact the code search steps.

Table 1: Change Distances, by detection capabilities and usage as latent index.

Edit Distance	Change Detection Ins/Del	Moved	Modified	Latent Index	References
diff	✓	✓	✓	×	[7, 4, 8, 9]
file changes	✓	✓[a]	✓[a]	×	[4, 9]
subtree matches	✓	✓	✓	×	[8, 9]
count	✓	×	×	✓	
paths	✓	✓	×	✓[b]	Our approach
contents	✓	×	✓	✓[b]	
paths + contents	✓	✓	✓	✓[b]	

[a] only on files [b] must be hashed to have a fixed sized

Scale by reducing the amount of code traversed,
only requiring to traverse nodes that lead to instances of
the searched patterns **or** changed code.

Early Results

Table 2: Code Query (Figure 1a) Runtime per snapshot

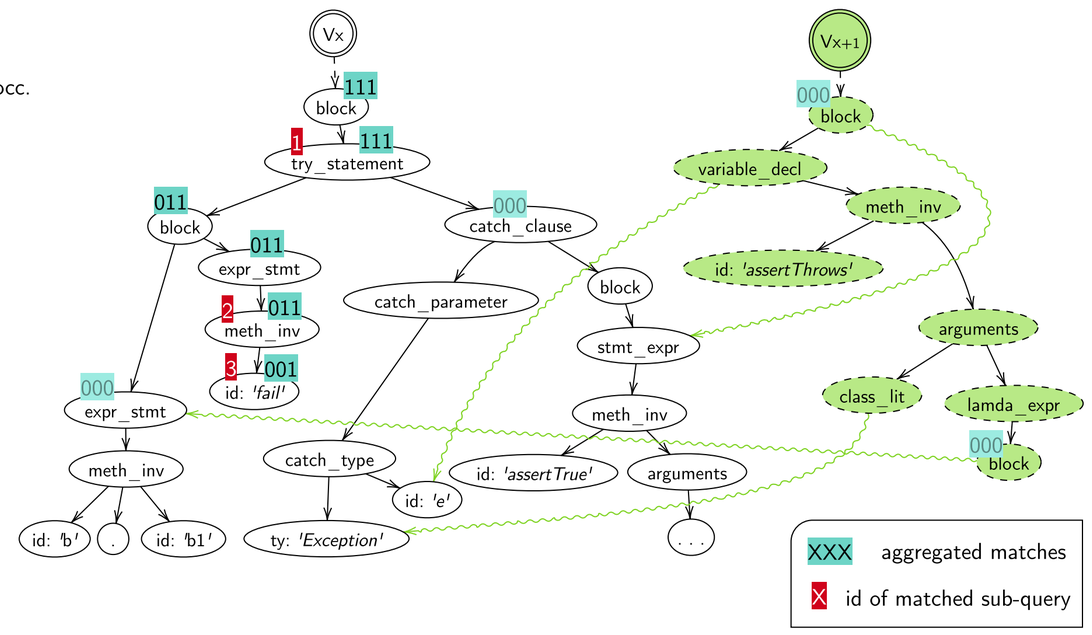
Projects	LoC	Runtime (ms) Mean	Min	Max	Matches Min	Max
hadoop	1.63M	719	685	982	2,579	2,600
flink	1.50M	Timeout	Timeout	>1s	Timeout	>918
quarkus	614K	Timeout	Timeout	>1s	Timeout	>63
guava	509k	29	21	44	0	0
netty	317k	127	114	499	150	194
dubbo	197k	146	100	321	82	128
fastjson	188k	14	9	21	1	2
logging-log4j2	183k	177	157	339	19	23
jenkins	181k	54	48	170	19	120
javaparser	179k	22	20	84	9	14
spoon	154k	66	56	188	71	73
aws-toolkit-eclipse	93.9k	7	0	15	0	0
maven	92.5k	197	181	409	0	51
spark	85.6k	154	141	355	2	43
skywalking	84.7k	254	70	618	1	2
jackson-core	52.3k	11	5	17	45	168
arthas	44.2k	24	17	63	0	0
jacoco	38.8k	44	26	92	0	0
junit4	31.2k	13	8	120	28	44
gson	25.8k	28	21	242	160	304
slf4j	13.5k	16	13	34	3	8

Example of a common refactoring (assertions on exceptions)

```
try {
  b.b1();
  fail();
} catch ( Missing e ) {
  assertTrue(
    e.getMessage()
     .startWith("Miss"));
}
```

```
var e = assertThrows(
  Missing.class,
  () -> { b.b1(); } );
assertTrue(
  e.getMessage()
   .startWith("Miss"));
```

The example stored in HyperAST



Query matching the example

Q1

```
(try_statement
  (block
    (statement
      (method_invocation
      (identifier) (#EQ? "fail"))))
  (catch_clause))
```

A Query can be made more general by relaxing contraints
e.g. removing patterns, abstracting node types

Example Sub-Queries

SQ1

```
(try_statement (block)
               (catch_clause) )
```

SQ2

```
(method_invocation
  (identifier) (#EQ? "fail") )
```

SQ3

```
(identifier) (#EQ? "fail")
```

SQ1 is obtained by removing content of the block in **Q1**
SQ2 is the content of the statement node in **Q1**
SQ3 is the content of the method_invocation node in **Q1**

Variant Queries

Q1 is also a sub-query of **Q2, Q3, Q4**

Q2

```
(try_statement
  (block
    (statement
      (method_invocation
      (identifier)
              (#EQ? "fail")))
    .
  )
  (catch_clause))
```

Q3

```
(try_statement
  (block
    .
    (statement)
    .
    (statement
      (method_invocation
      (identifier)
              (#EQ? "fail")))
    .
  )
  (catch_clause))
```

Q4

```
(try_statement
  (block
    (statement
      (method_invocation
      (identifier)
              (#EQ? "fail")))
    .
  )
  (catch_clause (_
    (catch_type
      (identifier)
              (#EQ? "Missing")))))
```

Possible benefits for common issues

Beyond Linear Histories:
considering evolutions from merged branches

commits can be squashed and cherry-picked,
finding merging metadata can be difficult and error prone

reducing the cost of snapshot comparisons offers opportunities to find more evolutions

Beyond Changed Files:
filtering fine-grained evolution components

some commits can modify many files,
breaking files based optimizations and accuracy

finer-grained code representations alleviate issues with worst cases of changes spread over entire codebases

Possible Directions

Synthethizing queries from examples
through a meta query system
queries themselves can be put in the HyperAST

Providing name resolution capabilities
adopting the scope-at-types formalism
provide def-uses

Handling high occurences patterns
extend the sub-queries to compute metrics

Find complex evolution trends
number of variant patterns over time

Prior works enbling the approach

HyperAST: Enabling Efficient Analysis of Software Histories at Scale
ASE 2022

HyperDiff: Computing Source Code Diffs at Scale
FSE 2023